Spam Email Classifier

Justin Kim

2026-03-07

Setup & data loading

```
library(ggplot2)
library(gridExtra)
library(MASS)
library(e1071)

# load in train and test sets
train_raw <- read.table("data/spam-train.txt", sep = ",",
header = FALSE)
test_raw  <- read.table("data/spam-test.txt",  sep = ",",
header = FALSE)

# cols 1-57 = features, col 58 = spam label (0 or 1)
X_train <- train_raw[, 1:57]
y_train <- train_raw[, 58]
X_test  <- test_raw[, 1:57]
y_test  <- test_raw[, 58]

# convert labels to factors for classification functions
y_train_fac <- factor(y_train, levels = c(0, 1), labels =
c("ham", "spam"))
y_test_fac  <- factor(y_test,  levels = c(0, 1), labels =
c("ham", "spam"))
```

Preprocessing

I'm trying three different transformations on the raw features to see which one gives classifiers the best shot at separating spam from ham.

Standardize (zero mean, unit var)

```
# important: fit stats on train only, then apply to both
col_means <- apply(X_train, 2, mean)
col_sds   <- apply(X_train, 2, sd)

X_train_std <- scale(X_train, center = col_means, scale =
col_sds)
X_test_std  <- scale(X_test,  center = col_means, scale =
col_sds)
```

Log transform

```
# log(x+1) -- the +1 keeps zeros from blowing up
X_train_log <- log(X_train + 1)
X_test_log  <- log(X_test + 1)
```

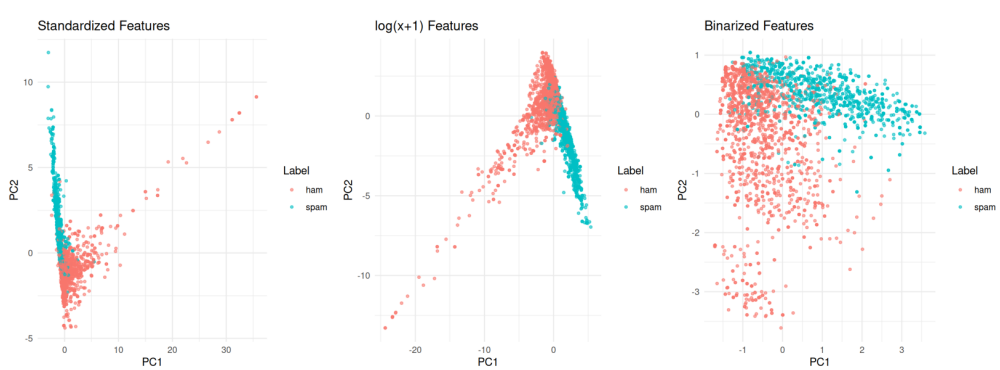
Binarize

```
# just checking if the feature is nonzero
X_train_bin <- as.data.frame((X_train > 0) * 1)
X_test_bin  <- as.data.frame((X_test > 0) * 1)
```

(a) PCA visualization

Reducing to 2 dimensions with PCA so I can actually see what's going on.

```
# standardized -- already centered/scaled so no need to r
edo
pca1 <- prcomp(X_train_std, center = FALSE, scale. = FALS
E)
df1 <- data.frame(PC1 = pca1$x[,1], PC2 = pca1$x[,2], Lab
el = y_train_fac)

p1 <- ggplot(df1, aes(PC1, PC2, color = Label)) +
  geom_point(alpha = 0.6, size = 1) +
  labs(title = "Standardized Features") +
  theme_minimal()

# log -- let prcomp handle centering/scaling
pca2 <- prcomp(X_train_log, center = TRUE, scale. = TRUE)
df2 <- data.frame(PC1 = pca2$x[,1], PC2 = pca2$x[,2], Lab
el = y_train_fac)

p2 <- ggplot(df2, aes(PC1, PC2, color = Label)) +
  geom_point(alpha = 0.6, size = 1) +
  labs(title = "log(x+1) Features") +
  theme_minimal()

# binary -- center but don't scale (scaling 0/1 vars is w
eird)
pca3 <- prcomp(as.data.frame(lapply(X_train_bin, as.numer
ic)),
               center = TRUE, scale. = FALSE)
df3 <- data.frame(PC1 = pca3$x[,1], PC2 = pca3$x[,2], Lab
el = y_train_fac)

p3 <- ggplot(df3, aes(PC1, PC2, color = Label)) +
  geom_point(alpha = 0.6, size = 1) +
  labs(title = "Binarized Features") +
  theme_minimal()

grid.arrange(p1, p2, p3, ncol = 3)
```



All three show some separation between spam and ham along the first two PCs, but the log-transformed version seems to give the cleanest clusters. The binarized features compress everything into a tight grid since every value is 0 or 1.

(b) Logistic regression

```
# threshold for bonferroni correction -- testing 57 predi
ctors
bonf_cutoff <- 0.05 / ncol(X_train_std)

# helper to fit logistic regression and return errors + s
ignificant vars
logreg <- function(xtr, ytr, xte, yte) {
  df_tr <- data.frame(xtr, y = ytr)
  df_te <- data.frame(xte, y = yte)

  fit <- glm(y ~ ., data = df_tr, family = binomial)

  # predict on both sets
  phat_tr <- predict(fit, type = "response") > 0.5
  phat_te <- predict(fit, newdata = df_te, type = "respon
se") > 0.5

  err_tr <- mean(phat_tr != (ytr == "spam"))
  err_te <- mean(phat_te != (yte == "spam"))

  # pull out p-values
  ctab <- summary(fit)$coefficients
  pvals <- ctab[, 4]

  list(
    train_err = err_tr,
    test_err  = err_te,
    sig_005   = names(pvals)[pvals < 0.05],
    sig_bonf  = names(pvals)[pvals < bonf_cutoff],
    coef_tab  = ctab
  )
}
```

```
lr_std <- logreg(X_train_std, y_train_fac, X_test_std, y_
test_fac)
lr_log <- logreg(X_train_log, y_train_fac, X_test_log, y_
test_fac)
lr_bin <- logreg(X_train_bin, y_train_fac, X_test_bin, y_
test_fac)
```

Classification errors

```
lr_errs <- data.frame(
  Preprocessing = c("Standardized", "Log-transformed", "B
inarized"),
  Train = c(lr_std$train_err, lr_log$train_err, lr_bin$tr
ain_err),
  Test  = c(lr_std$test_err,  lr_log$test_err,  lr_bin$te
st_err)
)
knitr::kable(lr_errs, digits = 3, col.names = c("Preproce
ssing", "Train Err", "Test Err"))
```

Preprocessing	Train Err	Test Err
Standardized	0.072	0.073
Log-transformed	0.058	0.057
Binarized	0.057	0.081

Significant predictors

```
cat("Standardized -- p < 0.05:\n",
    paste(lr_std$sig_005, collapse = ", "), "\n\n")
```

```
## Standardized -- p < 0.05:
##  (Intercept), V4, V5, V7, V8, V9, V11, V16, V17, V19,
V20, V21, V23, V25, V27, V42, V44, V45, V46, V47, V49, V5
2, V53, V55, V56, V57
```

```
cat("Standardized -- Bonferroni (p <", round(bonf_cutoff,
5), "):\n",
    paste(lr_std$sig_bonf, collapse = ", "), "\n\n")
```

```
## Standardized -- Bonferroni (p < 0.00088 ):
##  (Intercept), V5, V7, V16, V17, V20, V21, V23, V25, V2
7, V45, V46, V52, V53, V56, V57
```

```
cat("Log -- p < 0.05:\n",
    paste(lr_log$sig_005, collapse = ", "), "\n\n")
```

```
## Log -- p < 0.05:
##  (Intercept), V5, V7, V8, V11, V13, V16, V17, V20, V2
1, V23, V24, V25, V27, V28, V33, V35, V37, V42, V43, V45,
V46, V49, V52, V53, V57
```

```
cat("Log -- Bonferroni:\n",
    paste(lr_log$sig_bonf, collapse = ", "), "\n\n")
```

```
## Log -- Bonferroni:
##  (Intercept), V5, V7, V16, V17, V21, V23, V25, V27, V4
5, V46, V52, V53, V57
```

```
cat("Binarized -- p < 0.05:\n",
    paste(lr_bin$sig_005, collapse = ", "), "\n\n")
```

```
## Binarized -- p < 0.05:
##  (Intercept), V5, V7, V8, V10, V11, V13, V14, V15, V1
6, V17, V18, V20, V21, V23, V24, V25, V27, V28, V37, V42,
V43, V44, V45, V46, V48, V52, V53, V54
```

```
cat("Binarized -- Bonferroni:\n",
    paste(lr_bin$sig_bonf, collapse = ", "), "\n")
```

```
## Binarized -- Bonferroni:
##  (Intercept), V5, V7, V11, V13, V16, V17, V21, V23, V2
4, V25, V27, V28, V37, V42, V45, V46, V52, V53
```

Across all three, the log version had the lowest test error (~5.7%), then standardized (~7.3%), and binarized last (~8.1%). With 57 predictors, a bunch show up as significant at the unadjusted $p < 0.05$ level, but after Bonferroni correction ($\alpha^* \approx 0.00088$), only about a dozen survive. The ones that keep showing up regardless of preprocessing are V5, V7, V16, V17, V21, V23, V25, V27, V45, V46, V52, V53 – these are the heavy hitters for

spam detection (word frequencies like “free”, “remove”, “your” plus capital letter statistics). The binarized model also flags a few more (V11, V13, V24, V28, V37, V42), which makes sense since for a lot of these tokens, just knowing whether they appear at all carries most of the information.

(c) LDA & QDA

Only running these on standardized and log-transformed data. Binary features make QDA’s covariance estimation unstable so I left that out.

```
disc_analysis <- function(type, xtr, ytr, xte, yte) {
  df_tr <- data.frame(xtr, y = ytr)
  df_te <- data.frame(xte, y = yte)

  if (type == "lda") {
    mod <- lda(y ~ ., data = df_tr)
  } else {
    mod <- qda(y ~ ., data = df_tr)
  }

  pred_tr <- predict(mod, df_tr)$class
  pred_te <- predict(mod, df_te)$class

  c(train_error = mean(pred_tr != ytr),
    test_error  = mean(pred_te != yte))
}
```

```
lda_std_res <- disc_analysis("lda", X_train_std, y_train_
fac, X_test_std, y_test_fac)
qda_std_res <- disc_analysis("qda", X_train_std, y_train_
fac, X_test_std, y_test_fac)
lda_log_res <- disc_analysis("lda", X_train_log, y_train_
fac, X_test_log, y_test_fac)
qda_log_res <- disc_analysis("qda", X_train_log, y_train_
fac, X_test_log, y_test_fac)

da_errs <- data.frame(
  Method = c("LDA (Standardized)", "QDA (Standardized)",
             "LDA (Log-transformed)", "QDA (Log-transform
ed)"),
  Train = c(lda_std_res["train_error"], qda_std_res["trai
n_error"],
            lda_log_res["train_error"], qda_log_res["trai
n_error"]),
  Test  = c(lda_std_res["test_error"], qda_std_res["test_
error"],
            lda_log_res["test_error"], qda_log_res["test_
error"])
)
knitr::kable(da_errs, digits = 3, col.names = c("Method",
"Training Error", "Test Error"))
```

Method	Training Error	Test Error
LDA (Standardized)	0.102	0.096
QDA (Standardized)	0.179	0.184
LDA (Log-transformed)	0.060	0.065
QDA (Log-transformed)	0.159	0.157

QDA does noticeably worse than LDA here – probably overfitting because it has to estimate a full 57×57 covariance matrix per class and there aren't enough samples to do that reliably.

(d) SVM

Testing linear and RBF kernels on all three preprocessed versions.

Setup & data loading

Preprocessing

(a) PCA visualization

(b) Logistic regression

(c) LDA & QDA

(d) SVM

All results

Conclusions

```
fit_svm <- function(kern, xtr, ytr, xte, yte) {
  df_tr <- data.frame(xtr, y = ytr)
  df_te <- data.frame(xte, y = yte)

  mod <- svm(y ~ ., data = df_tr,
             kernel = kern,
             cost = 1,
             gamma = 1 / ncol(xtr),
             scale = FALSE)

  p_tr <- predict(mod, df_tr)
  p_te <- predict(mod, df_te)

  c(train_error = mean(p_tr != ytr),
    test_error  = mean(p_te != yte))
}
```

```
data_versions <- list(
  Std = list(tr = X_train_std, te = X_test_std),
  Log = list(tr = X_train_log, te = X_test_log),
  Bin = list(tr = X_train_bin, te = X_test_bin)
)

svm_res <- do.call(rbind, lapply(names(data_versions), fu
nction(v) {
  d <- data_versions[[v]]
  lin <- fit_svm("linear", d$tr, y_train_fac, d$te, y_tes
t_fac)
  rbf <- fit_svm("radial", d$tr, y_train_fac, d$te, y_tes
t_fac)

  data.frame(
    Preprocessing = v,
    Kernel = c("Linear", "RBF"),
    Train  = c(lin["train_error"], rbf["train_error"]),
    Test   = c(lin["test_error"],  rbf["test_error"])
  )
}))

knitr::kable(svm_res, digits = 3,
             col.names = c("Preprocessing", "Kernel", "Tr
ain Err", "Test Err"))
```

Preprocessing	Kernel	Train Err	Test Err
Std	Linear	0.065	0.068

Std	RBF	0.052	0.065
Log	Linear	0.058	0.056
Log	RBF	0.060	0.057
Bin	Linear	0.060	0.074
Bin	RBF	0.062	0.076

All results

```
# clean up svm results for merging
svm_fmt <- svm_res
svm_fmt$Method <- paste("SVM", svm_fmt$Kernel)
svm_fmt$Data   <- svm_fmt$Preprocessing
svm_fmt <- svm_fmt[, c("Method", "Data", "Train", "Tes
t")]

everything <- rbind(
  data.frame(Method = "Logistic", Data = "Standardized",
             Train = lr_std$train_err, Test = lr_std$test
_err),
  data.frame(Method = "Logistic", Data = "Log",
             Train = lr_log$train_err, Test = lr_log$test
_err),
  data.frame(Method = "Logistic", Data = "Binarized",
             Train = lr_bin$train_err, Test = lr_bin$test
_err),
  data.frame(Method = "LDA", Data = "Standardized",
             Train = as.numeric(lda_std_res["train_erro
r"]),
             Test  = as.numeric(lda_std_res["test_erro
r"])),
  data.frame(Method = "QDA", Data = "Standardized",
             Train = as.numeric(qda_std_res["train_erro
r"]),
             Test  = as.numeric(qda_std_res["test_erro
r"])),
  data.frame(Method = "LDA", Data = "Log",
             Train = as.numeric(lda_log_res["train_erro
r"]),
             Test  = as.numeric(lda_log_res["test_erro
r"])),
  data.frame(Method = "QDA", Data = "Log",
             Train = as.numeric(qda_log_res["train_erro
r"]),
             Test  = as.numeric(qda_log_res["test_erro
r"])),
  svm_fmt
)

knitr::kable(everything, digits = 3,
             col.names = c("Method", "Data", "Train Erro
r", "Test Error"))
```

Method	Data	Train Error	Test Error

Logistic	Standardized	0.072	0.073
Logistic	Log	0.058	0.057
Logistic	Binarized	0.057	0.081
LDA	Standardized	0.102	0.096
QDA	Standardized	0.179	0.184
LDA	Log	0.060	0.065
QDA	Log	0.159	0.157
SVM Linear	Std	0.065	0.068
SVM RBF	Std	0.052	0.065
SVM Linear	Log	0.058	0.056
SVM RBF	Log	0.060	0.057
SVM Linear	Bin	0.060	0.074
SVM RBF	Bin	0.062	0.076

Conclusions

The linear SVM on log-transformed data gave the best test error at 0.056, with the RBF SVM on the same data right behind at ~0.057. The fact that a linear kernel does this well tells me the classes are mostly linearly separable once the features are log-transformed. Train error (0.058) and test error (0.056) are close, so the model isn't overfitting – that's reassuring.

Still, ~5% misclassification means roughly 1 in 20 emails gets the wrong label. There's room for improvement with hyperparameter tuning, ensembles, or better feature engineering.